SOFTWARE CONFIGURATION MANAGEMENT

Name: Mohamed Ahmed Mohamed Group:SE1

Task 1: Repository Inspection and VCS Understanding (10 Marks)

1.1 Repository State Information

Item	Your Finding / Terminal Output
Current branch	main
Number of visible branches	3 (main, feature-search, feature-stock-validation)
Names of feature branches	feature-search, feature-stock-validation
Latest commit message on main	Add project changelog

```
PS C:\Users\Mohammed\Desktop\Project\student-repo> git status
>> git branch
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        src/__pycache__/

nothing added to commit but untracked files present (use "git add" to track)
  feature-search
  feature-stock-validation
* main
PS C:\Users\Mohammed\Desktop\Project\student-repo> git log --oneline --graph --all --decorate
* 6aa0271 (feature-search) Add product search feature
| * e048321 (feature-stock-validation) Add stock quantity validation feature
|/
* 812bde2 (HEAD -> main) Add project changelog
* 72ca347 Update monitor price
* 50b3f36 Update stock module notes
* d1d5c34 Adjust stock validation behavior
* bfd0bf1 Add product display helper
* 52b5e8c Add mouse product and stock
* bfa0f80 Document prototype status
* c408215 Initial inventory project
```

1.2 VCS Conceptual Questions

1. What is the main difference between a centralized VCS and a distributed VCS?
Answer: In a Centralized Version Control System (CVCS), a single central server holds the master repository and full version history; developers connect to this central node to pull working copies and push changes. In contrast, a Distributed Version Control System (DVCS) like Git provides every developer with a full, independent clone of the complete repository, including its entire historical commit graph and branches.

2. Why is a distributed VCS less dependent on one central repository for local work?
Answer: Because each local clone possesses the entire database and history locally, developers can perform virtually all essential operations—such as creating commits, branching, switching context, reviewing history, and diffing code versions—completely offline and without network access or dependence on a single remote server.

3. What is the purpose of creating a feature branch?
Answer: A feature branch isolates new functionality, bug fixes, or experimental work away from the stable production branch (main). This allows for concurrent development among multiple team members without destabilizing the main codebase until code review and testing are completed.

Task 2: Branching, Merging, and Conflict Resolution (20 Marks)

2.1 Merge Conflict Evidence (Before Resolution)

```
PS C:\Users\Mohammed\Desktop\Project\student-repo> git merge feature-stock-validation
Auto-merging version.txt
CONFLICT (content): Merge conflict in version.txt
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\Mohammed\Desktop\Project\student-repo>
```

```
<<<<<<< HEAD
Version: 2.0-search
=======
Version: 2.0-stock
>>>>>>> feature-stock-validation
```

2.2 Corrected version.txt Content

Version: 2.0

```
Version: 2.0
```

2.3 Post-Merge Verification & Commit Graph

```
PS C:\Users\Mohammed\Desktop\Project\student-repo> Get-ChildItem src


    Directory: C:\Users\Mohammed\Desktop\Project\student-repo\src


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         10/7/2026   1:16 AM                __pycache__
-a----         10/7/2026   1:16 AM            499 inventory.py
-a----        10/18/2026  12:42 PM            163 search.py
-a----        10/18/2026  12:46 PM            120 stock_validation.py


PS C:\Users\Mohammed\Desktop\Project\student-repo> git log --oneline --graph --decorate -n 6
*   d97a64f (HEAD -> main) Merge feature-stock-validation and resolve version conflict to 2.0
|\
| * e048321 (feature-stock-validation) Add stock quantity validation feature
* | 6aa0271 (feature-search) Add product search feature
|/
* 812bde2 Add project changelog
* 72ca347 Update monitor price
* 50b3f36 Update stock module notes
PS C:\Users\Mohammed\Desktop\Project\student-repo>
```

Question: Why could Git not automatically choose between the two version numbers?
Answer: Both 'feature-search' and 'feature-stock-validation' diverged from the exact same base commit on main, and both branches modified the exact same line within 'version.txt' independently. Because both commits introduced conflicting edits to the exact same location since their common ancestor, Git has no deterministic, semantic way to decide which version takes precedence, intentionally halting to require human resolution.

Task 3: Find and Correct a Defect Using Git History (20 Marks)

3.1 Test Failure Output

```
PS C:\Users\Mohammed\Desktop\Project\student-repo> python test_inventory.py
PASS: positive stock is valid
PASS: zero stock is valid
FAIL: negative stock must be rejected (expected False, got True)
PS C:\Users\Mohammed\Desktop\Project\student-repo>
```

```
PS C:\Users\Mohammed\Desktop\Project\student-repo> git log --oneline
d97a64f (HEAD -> main) Merge feature-stock-validation and resolve version conflict to 2.0
6aa0271 (feature-search) Add product search feature
e048321 (feature-stock-validation) Add stock quantity validation feature
812bde2 Add project changelog
72ca347 Update monitor price
50b3f36 Update stock module notes
d1d5c34 Adjust stock validation behavior
bfd0bf1 Add product display helper
52b5e8c Add mouse product and stock
bfa0f80 Document prototype status
c408215 Initial inventory project
PS C:\Users\Mohammed\Desktop\Project\student-repo> git show d1d5c34
commit d1d5c34e01489d9a345602d95c31998b5123c081
Author: SCM Lab Author <scm-lab@example.edu>
Date:   Tue Oct 6 22:16:45 2026 +0000

    Adjust stock validation behavior

diff --git a/src/inventory.py b/src/inventory.py
index 8f724c0..8e1880c 100644
--- a/src/inventory.py
+++ b/src/inventory.py
@@ -16,6 +16,8 @@ def list_products():


 def stock_is_valid(quantity):
+    if quantity < 0:
+        return True
     return quantity >= 0


PS C:\Users\Mohammed\Desktop\Project\student-repo> git diff bfd0bf1 d1d5c34
diff --git a/src/inventory.py b/src/inventory.py
index 8f724c0..8e1880c 100644
--- a/src/inventory.py
+++ b/src/inventory.py
@@ -16,6 +16,8 @@ def list_products():


 def stock_is_valid(quantity):
+    if quantity < 0:
+        return True
     return quantity >= 0


PS C:\Users\Mohammed\Desktop\Project\student-repo>
```

3.2 Defect Investigation Table

Investigation Item	Your Answer / Findings
Failing file / function	src/inventory.py / stock_is_valid(quantity)
Commit hash that introduced defect	d1d5c34
Commit message	Adjust stock validation behavior
Incorrect behavior	When quantity < 0, it returns True, thereby validating and accepting negative stock.
Correct behavior	When quantity < 0, it must reject negative stock (return False); it should only return True for zero or positive values (quantity >= 0).

3.3 Defect Correction Commit & Passing Test Output

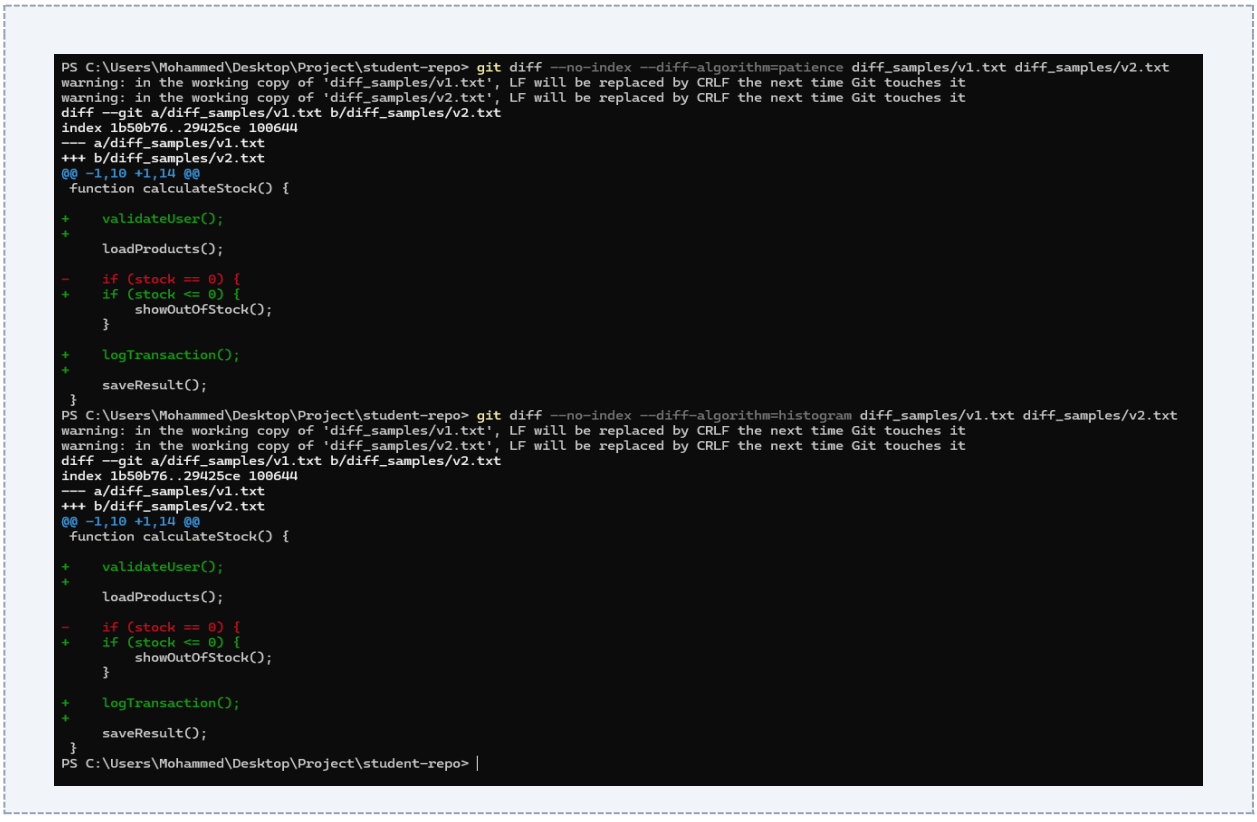
```
PS C:\Users\Mohammed\Desktop\Project\student-repo> python test_inventory.py
PASS: positive stock is valid
PASS: zero stock is valid
PASS: negative stock must be rejected
PS C:\Users\Mohammed\Desktop\Project\student-repo>
```

Task 4: Git Diff Algorithm Comparison (15 Marks)

4.1 Algorithm Comparison Table

Algorithm	Easy to read?	Main Observation	Ranking (1-4)
Myers	Moderate	Default greedy algorithm; finds the shortest	3

		edit script but can split semantic blocks or matching brackets awkwardly across hunk boundaries.	
Minimal	Moderate	Spends additional computation to find strictly minimal line differences; output is often virtually identical to Myers for small sample files.	4
Patience	Yes	Matches unique common lines first (like function signatures); avoids matching isolated brackets, producing clean, logically grouped hunks.	1
Histogram	Yes	An optimized, high-performance variant of patience; handles repetitive and low-occurrence lines cleanly, keeping functions readable.	2



```
PS C:\Users\Mohammed\Desktop\Project\student-repo> git diff --no-index --diff-algorithm=patience diff_samples/v1.txt diff_samples/v2.txt
warning: in the working copy of 'diff_samples/v1.txt', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'diff_samples/v2.txt', LF will be replaced by CRLF the next time Git touches it
diff --git a/diff_samples/v1.txt b/diff_samples/v2.txt
index 1b50b76..29425ce 100644
--- a/diff_samples/v1.txt
+++ b/diff_samples/v2.txt
@@ -1,10 +1,14 @@
 function calculateStock() {

+    validateUser();
+
     loadProducts();

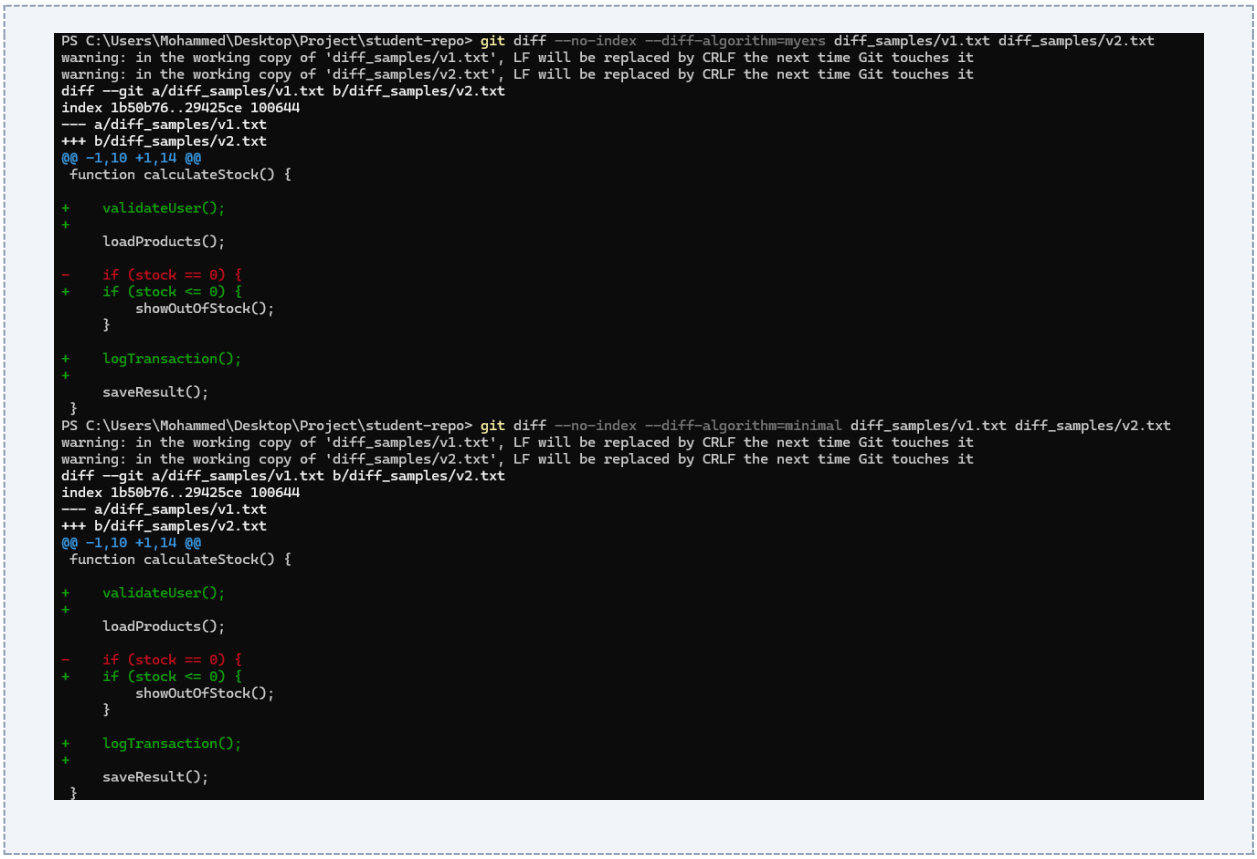
-    if (stock == 0) {
+    if (stock <= 0) {
         showOutOfStock();
     }

+    logTransaction();
+
     saveResult();
 }
PS C:\Users\Mohammed\Desktop\Project\student-repo> git diff --no-index --diff-algorithm=histogram diff_samples/v1.txt diff_samples/v2.txt
warning: in the working copy of 'diff_samples/v1.txt', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'diff_samples/v2.txt', LF will be replaced by CRLF the next time Git touches it
diff --git a/diff_samples/v1.txt b/diff_samples/v2.txt
index 1b50b76..29425ce 100644
--- a/diff_samples/v1.txt
+++ b/diff_samples/v2.txt
@@ -1,10 +1,14 @@
 function calculateStock() {

+    validateUser();
+
     loadProducts();

-    if (stock == 0) {
+    if (stock <= 0) {
         showOutOfStock();
     }

+    logTransaction();
+
     saveResult();
 }
PS C:\Users\Mohammed\Desktop\Project\student-repo>
```



Which diff output was easiest for you to understand, and why? (3-5 sentences)
Answer: The Patience diff algorithm produced the easiest output to understand. While the standard Myers and Minimal algorithms focus strictly on finding the mathematically shortest edit script, they frequently chunk up identical, non-unique syntax—such as closing curly braces or blank lines—into unrelated code sections. In contrast, Patience first identifies unique common anchors (like method definitions), preserving the complete structural integrity of functions. This semantic alignment prevents fragmented code hunks and accurately illustrates developer intent.

Task 5: Distributed Collaboration: Commit, Push, and Pull (15 Marks)

5.1 Collaboration Lifecycle Table

Stage	Stored in A locally?	On remote?	Visible in B?
After edit	Working Tree only (untracked / modified)	No	No
After commit	Yes (Committed in local .git repo)	No	No

After push	Yes (Local repo updated)	Yes (Reflected in remote.git)	No (Not yet pulled)
After pull in B	Yes	Yes	Yes (Synchronized to B's working tree)

```
PS C:\Users\Mohammed\Desktop\Project\collaboration\developer-B> cd ../developer-A
PS C:\Users\Mohammed\Desktop\Project\collaboration\developer-A> git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 368 bytes | 368.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
To ../remote.git
   812bde2..34baa6b  main -> main
PS C:\Users\Mohammed\Desktop\Project\collaboration\developer-A> cd ../developer-B
PS C:\Users\Mohammed\Desktop\Project\collaboration\developer-B> git pull origin main
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 348 bytes | 43.00 KiB/s, done.
From ../remote
 * branch            main       -> FETCH_HEAD
   812bde2..34baa6b  main       -> origin/main
Updating 812bde2..34baa6b
Fast-forward
 README.md | Bin 298 -> 386 bytes
 1 file changed, 0 insertions(+), 0 deletions(-)
PS C:\Users\Mohammed\Desktop\Project\collaboration\developer-B> Get-Content README.md
# Inventory Management System

A small teaching project used for the Software Configuration Management lab.

Current modules:
- Product listing
- Stock checking

Use Git history and branches to complete the lab tasks.

Project status: classroom prototype.

The stock module is under active review.
Team note: Inventory validation completed.
```

Question: Why does git commit alone not make Developer A's change visible to Developer B?
Answer: In a distributed version control system, 'git commit' is a completely localized command that records changes exclusively to Developer A's private repository on their local drive. No automatic network synchronization occurs. The commit remains isolated until Developer A explicitly transfers the refspec to the shared remote repository via 'git push', and Developer B queries and incorporates those updates into their own local repository via 'git pull'.

Task 6: Git Integrity and SSH Concepts (10 Marks)

1. Record the short hash of your latest commit. After making a new commit, does the new commit have the same hash or a different one?

Latest Commit Short Hash: e2203fb

Answer: The new commit will have a completely different hash. Git calculates commit hashes cryptographically based on the commit's content tree, the parent commit's hash, author metadata, timestamp, and commit message. Any modification or addition inherently alters the input data, resulting in a distinct new cryptographic hash.

2. The lecture describes SHA-1 as producing 160 bits (20 bytes), usually displayed as 40 hexadecimal characters. Explain why 160 bits correspond to 40 hexadecimal digits.

Answer: A single hexadecimal digit represents 16 possible values (0-9 and a-f), which corresponds to 4 bits of binary information (2^4 = 16). Dividing the total number of bits by the number of bits represented by each hexadecimal character yields exactly 40 characters: 160 bits / 4 bits per digit = 40 hexadecimal digits.

3. What is the purpose of hashing in a Git repository?

Answer: Cryptographic hashing provides content-addressable storage and guarantees data integrity. In Git, every blob, tree, commit, and annotated tag is identified uniquely by its hash. This makes the repository history completely tamper-evident: any unauthorized modification, corruption, or change to a file or historical commit cascades through the DAG (Directed Acyclic Graph) and changes the resulting hash, making silent alterations impossible.

4. Which SSH key may be shared with a remote service: the public key or the private key?

Answer: The public key (e.g., id_ed25519.pub or id_rsa.pub) is the only key that may be safely shared with and uploaded to remote hosting services.

5. Why must the private SSH key be kept secret?

Answer: The private SSH key serves as the confidential cryptographic proof of identity. If an unauthorized party obtains the private key, they can fully impersonate the user, bypass authentication barriers, and sign, push, or alter code repositories under that developer's credentials.

Task 7: Final Report Reflection (10 Marks)

Comprehensive SCM Reflection:

Version control systems form the operational backbone of modern software configuration management by ensuring traceability, stability, and integrity across distributed engineering workflows. Git's branching model isolates features, hotfixes, and parallel experiments, allowing developers to work concurrently without destabilizing the production baseline on main. When branches converge, merge mechanisms and conflict markers mandate intentional reconciliation rather than accidental overwrites. Furthermore, Git's distributed architecture decouples local developer productivity from central server uptime, granting every contributor an immutable, complete repository history for rapid auditing, diffing, and defect localization via historical logs. Beneath the interface, SHA hashing establishes content-addressable storage where every artifact is tamper-evident, ensuring that neither code nor metadata can be altered undetected. Combined with

asymmetric SSH cryptography for secure identity verification and access control, Git guarantees an auditable, secure, and resilient configuration baseline essential for continuous integration, collaboration, and high-integrity software delivery.